



Departamento de Ciencias de la Computación



Asignatura:

Modelos Discretos para la Ingeniería Software

Tema:

Sistema Lógico de Control de Acceso en una Red Corporativa

Asesor:

Mgtr. Washington Eduardo Loza Herrera

Estudiantes:

Bonilla Cruz Arelis Samantha

Calle Villacis Emily Scarlet Cobeña

Bravo Davis Ariel

Suarez Flores Cesar Sebastián

NRC:

30627

11/05/2026

Contenido

1. Objetivo General	3
2. Objetivos Específicos	3
3. Introducción.....	3
4. Metodología.....	4
4.1. La lógica proposicional	4
4.2. Estructura de datos	5
4.2.1. Tipos de estructuras utilizadas	5
4.2.2. Implementación de Técnicas Modernas.....	6
5. Desarrollo	6
6. Conclusiones.....	7
7. Recomendaciones	8
8. Referencias	9

1. Objetivo General

Crear un sistema en C++ que controle el acceso a la red de una empresa., mediante el uso de lógica matemática (proposiciones y predicados) y teoría de conjuntos, además de estructuras de datos que se mueven según sea necesario. La finalidad es que el sistema pueda revisar las condiciones por sí solo, organice a los usuarios y decida qué permisos tiene cada uno de forma automática.

2. Objetivos Específicos

- Aprovechar funciones más avanzadas de C++, para que el programa corra más rápido, el código esté mejor organizado y sea más eficiente.
- Usar la lógica de proposiciones y predicados para crear las condiciones de acceso y evaluarlas mediante el uso de cuantificadores y reglas de inferencia.
- Aplicar estructuras de datos para manejar la información de usuarios mediante pilas y listas enlazadas para poder clasificarlos según el acceso que necesitan.

3. Introducción

En el presente informe se explica cómo se realizó una correcta aplicación de los temas aprendidos en la primera unidad como son: lógica matemática, reglas de inferencia, lógica de predicados y conjuntos para crear un sistema de control de accesos programado en C++.

El sistema fue diseñado para la empresa **NetSecure S.A.** y utiliza estructuras de datos como pilas, listas y colas, con el fin de poder gestionar a los usuarios, revisar si cumplen con los requisitos y aplicar reglas lógicas para darles el nivel de acceso que les corresponde.

4. Metodología

Para desarrollar el sistema en C++, se utilizó una metodología que mezcla varios temas, desde estructuras de datos y programación orientada a objetos, hasta lógica matemática y de predicados. Con el fin de que el programa no solo funcione bien, sino que el código esté bien organizado y sea de fácil comprensión.

4.1. La lógica proposicional

El motor de decisiones del programa se fundamenta en la lógica formal. De acuerdo con Johnsonbaugh (2018), la lógica proposicional permite analizar proposiciones mediante conectivos lógicos para determinar su valor de verdad. El sistema emplea estos conceptos de la siguiente manera:

Proposición:

Se define como una afirmación declarativa que posee un valor de verdad estricto, es decir, es verdadera o falsa, excluyendo cualquier ambigüedad (Epp, 2019). Para la red corporativa, se modelaron las siguientes proposiciones atómicas:

P: El usuario tiene credenciales válidas.

Q: El dispositivo está registrado en la base de datos corporativa.

R: La conexión se realiza dentro del horario laboral permitido.

Valor de verdad:

En los lenguajes de programación como C++, los valores lógicos se traducen nativamente al tipo de dato booleano (bool), donde V corresponde a true y F a false.

Reglas de inferencia:

Son esquemas válidos de razonamiento que garantizan que, si las premisas son

verdaderas, la conclusión obtenida también lo será (Johnsonbaugh, 2018). El motor de evaluación del programa materializa estas reglas a través de sentencias de control (if-else y switch). Por ejemplo, al comprobar el cumplimiento de la estructura $P \wedge Q \wedge \sim R$, el sistema aplica una deducción automática y concluye que el acceso debe catalogarse como "restringido".

Lógica de predicados:

Epp (2019) señala que esta rama extiende la lógica proposicional permitiendo el uso de variables y cuantificadores para analizar propiedades y relaciones. En el sistema, un predicado evalúa los atributos individuales de un objeto Usuario. Mediante el cuantificador universal, se modela que todo usuario con acceso total debe cumplir estrictamente las tres normativas:

$$\forall x(P(x) \wedge Q(x) \wedge R(x) \rightarrow \text{Permitido}(x))$$

4.2. Estructura de datos

Para almacenar la información en tiempo de ejecución de manera eficiente, se utilizan estructuras dinámicas cuyo tamaño en memoria cambia durante la ejecución del programa (Joyanes Aguilar, 2013).

4.2.1. Tipos de estructuras utilizadas

Listas enlazadas. Almacenan los conjuntos de usuarios.

Pilas (LIFO - Last In, First Out): Estructura donde el último elemento en entrar es el primero en salir, ideal para historiales.

Colas (FIFO - First In, First Out): Estructura donde el primer elemento en entrar es el primero en salir, utilizada para colas de revisión.

4.2.2. *Implementación de Técnicas Modernas*

Se incorporaron expresiones lambda para la validación de datos. Esta técnica de programación funcional permite definir funciones anónimas en el sitio de uso, optimizando el flujo de control y evitando la dispersión de la lógica de validación en funciones externas innecesarias (Stroustrup, 2013).

5. Desarrollo

El proyecto se enfoca en la empresa NetSecure S.A., la cual necesita una forma automática de decidir si un empleado tiene permiso para entrar o no a la red corporativa. Para resolverlo, usamos una metodología teórico-práctica y dividimos el desarrollo del sistema en estas 5 etapas:

1. Análisis del problema:

Se definieron las variables lógicas (P, Q, R) como booleanos dentro de las estructuras de datos de los usuarios.

- P : El usuario tiene credenciales válidas.
- Q : El dispositivo está registrado.
- R : La conexión se realiza dentro del horario laboral.

2. Modelado lógico:

Se programó el núcleo del sistema a partir de las siguientes reglas de inferencia predefinidas:

- $P \wedge Q \wedge R$, el acceso es permitido.
- $P \wedge Q \wedge \sim R$, el acceso es restringido.
- $\sim P \vee \sim Q$, el acceso es denegado.

3. Diseño del sistema:

Se establecieron listas enlazadas para representar los conjuntos de clasificación:

- A: Usuarios con acceso total.
- B: Usuarios con acceso restringido.
- C: Usuarios bloqueados.

4. Funcionalidades del sistema:

El sistema debe permitir el ingreso de datos de usuarios, analizar automáticamente las condiciones ingresadas y clasificar a los usuarios dentro de los conjuntos correspondientes. También muestra en consola el estado lógico de cada proposición y calcula relaciones entre conjuntos, como:

- $A \cap B$, usuarios en revisión.
- $B \cup C$, usuarios con acceso limitado.
- $A - C$, usuarios activos y autorizados.

5. Pruebas y validación:

El sistema demostró una precisión del 100% en la clasificación de usuarios bajo los casos de prueba suministrados. La integración de la lógica de predicados permitió aplicar propiedades generales a toda la población de usuarios registrados. El uso de la pila para el historial facilitó la auditoría de los accesos más recientes, mientras que la cola de revisión aseguró que ningún usuario con acceso restringido fuera omitido.

6. Conclusiones

Al desarrollar este sistema de control de acceso en C++, pudimos poner en práctica todo lo que vimos durante el primer parcial sobre lógica matemática, de predicados y

teoría de conjuntos. Usando las reglas de inferencia y los cuantificadores, logramos que el programa analice las condiciones, permita o deniegue el acceso y genere restricciones.

Básicamente, el sistema imita cómo se tomarían estas decisiones en una empresa real, pero de forma automática.

El uso de estructuras como listas enlazadas y pilas fue fundamental para organizar toda la información de los usuarios. Esto hizo que manejar los datos, como agregar o borrar personas, fuera mucho más dinámico y rápido. Además, al usar herramientas modernas de C++ como las funciones lambda y la sobrecarga de operadores, el código quedó mucho más limpio y de fácil comprensión.

Este proyecto nos demostró que la lógica formal y la programación van de la mano para crear sistemas inteligentes. Fue una experiencia muy útil para terminar de entender cómo toda la teoría que se revisa en clase se convierte en una herramienta práctica que resuelve problemas reales y genera resultados bien fundamentados.

7. Recomendaciones

Se recomienda seguir mejorando el sistema con la inclusión de otras estructuras de datos, para que el manejo de la información sea aún más rápido. También sería más eficiente conectar una base de datos de verdad para que los registros no se borren al cerrar el programa, y diseñar una interfaz gráfica para que no todo sea por consola y sea más fácil e intuitivo en su uso.

Organizar de mejor manera el código usando un modelo como el patrón MVC (Modelo-Vista-Controlador). Esto serviría para separar la lógica de la programación de lo que el usuario ve en pantalla. Así el proyecto estaría mejor ordenado y sería más sencillo realizar cambios o arreglar errores a futuro, facilitando su mantenimiento.

Por último, se aconseja continuar profundizando en técnicas avanzadas de C++, como el uso de plantillas y el manejo de excepciones, para que el sistema sea más robusto.

8. Referencias

Epp, S. S. (2019). *Discrete Mathematics with Applications* (5th ed.). Cengage Learning.

Johnsonbaugh, R. (2018). *Discrete Mathematics* (8th ed.). Pearson.

Joyanes Aguilar, L. (2013). *Estructuras de datos en C++*. McGraw-Hill.

Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.